

Coordenação

STD29006 – Engenharia de Telecomunicações

Prof. Emerson Ribeiro de Mello

mello@ifsc.edu.br

Licenciamento



Slides licenciados sob [Creative Commons "Atribuição 4.0 Internacional"](https://creativecommons.org/licenses/by/4.0/)

Coordenação

- Dado um conjunto de processos, como coordenar suas ações ou como garantir que acordem sobre um ou mais valores?

Coordenação em sistemas distribuídos deve considerar

- Possibilidade falhas de nós ou rede
- Não possui um coordenador fixo
- Não possui memória compartilhada
- Não possui relógio compartilhado

Formas de coordenação em sistemas distribuídos

■ Exclusão mútua

- Determina quem poderá entrar em uma seção crítica

■ Eleição de um líder

- Determina quem será o líder

■ Comunicação em grupo

- Mensagens são processadas na mesma ordem por todos os nós

Exclusão mútua

Sincronização e coordenação de processos

Sistemas Operacionais

Condições de corrida (*race conditions*)

Dois ou mais processos acessam dados compartilhados e o resultado dependerá da ordem de execução

- Como evitar condições de corrida?
 - Sincronizar processos e forçar uma ordem de execução
 - Uso de mecanismos de **exclusão mútua**, de forma que somente um processo entrará na **seção crítica** em um dado momento

Sincronização e coordenação de processos

Sistemas Operacionais

Seção crítica

Trecho de código onde se acessa **recurso compartilhado** o qual **não permite acesso concorrente**



- Nenhum processo que executa fora da seção crítica pode bloquear outro
- Nenhum processo esperará eternamente para entrar na seção crítica ou ficará por lá eternamente

Sincronização e coordenação de processos

Sistemas Operacionais

Deadlock

Dois ou mais processos aguardam por um evento que só pode ser gerado por algum destes processos que estão em espera



- Condições para ocorrência
 - Exclusão mútua
 - Posse e espera
 - Não preempção
 - Espera circular

Exclusão mútua em sistemas distribuídos I

- Processos precisam de acesso exclusivo a recursos compartilhados.
- Diferente de Sistemas Operacionais: não há variáveis compartilhadas, semáforos ou monitores.

Exclusão mútua em sistemas distribuídos II

Soluções para exclusão mútua em sistemas distribuídos:

- **Baseada em permissões:** Um processo solicita permissão a outros processos para acessar a seção crítica
- **Baseada em tokens:** O acesso à seção crítica é concedido ao processo que possui um token, que circula entre os processos

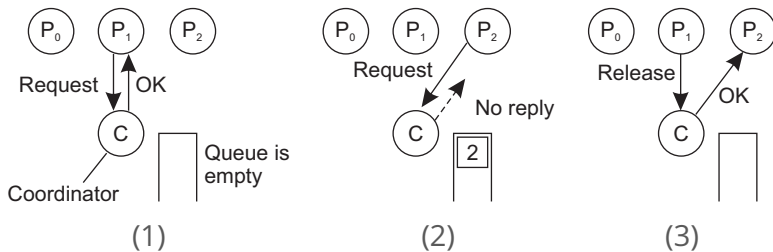
Exclusão mútua em sistemas distribuídos III

Tipos de algoritmos:

- Algoritmos centralizados
- Algoritmos descentralizados
- Algoritmos distribuídos
- Algoritmos baseados em anel (token ring)

Baseada em permissões

Algoritmo centralizado



- 1 Processo P_1 solicita acesso ao coordenador e a permissão é concedida
- 2 Processo P_2 solicita acesso ao coordenador para acessar o mesmo recurso e o coordenador não responde
- 3 Processo P_1 libera o recurso, avisa o coordenador o qual responde e concede a permissão para P_2

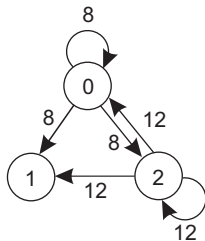
Baseada em permissões

Algoritmo descentralizado

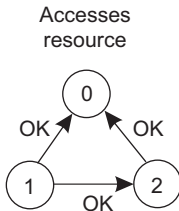
- Cada recurso **R** é replicado **N** vezes
- Cada réplica possui seu próprio coordenador
- Quando processo deseja entrar na seção crítica
 - 1 Envia pedido a todos os coordenadores indicando o recurso que deseja acessar
 - 2 Cada coordenador responde com OK ou NOK
- Processo ingressa somente se receber a maioria das mensagens OK
 - $respostas > N/2$
- Se a maioria das respostas for NOK, então o processo avisa os coordenadores que deram OK, indicando que ele não ganhou o acesso
 - Espera um tempo aleatório para tentar novamente

Baseada em permissões

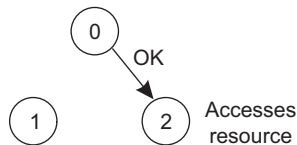
Distribuído – Ricard & Agrawala



(1)



(2)

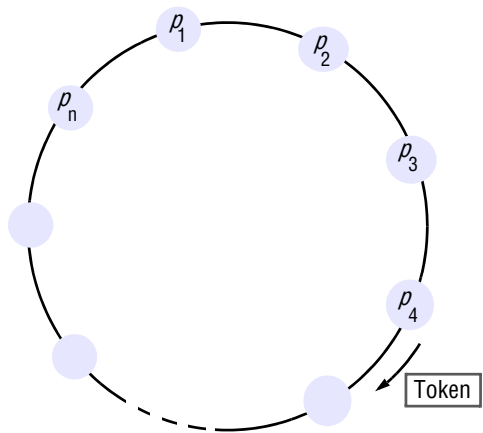


(3)

- 1 P_0 e P_2 desejam acessar um recurso compartilhado no mesmo instante
- 2 P_0 tem um relógio menor, assim ele ganha o acesso
- 3 Assim que P_0 termina, ele envia OK para P_2 e esse poderá acessar o recurso

Baseada em tokens

Token ring



- Grupo de N processos
 - P_i comunica com P_{i+1}
- Token circula pelo anel
 - De P_i para $P_{i+1} \bmod N$
- P_i ao receber o token
 - Se não desejar entrar na seção crítica, encaminha o token para o P_{i+1}
 - Caso contrário, mantém o token pelo tempo que permanecer na seção crítica
 - Libera o token depois de finalizado

Comparação entre os algoritmos

Algoritmo	Número de msgs para entrar/sair	Atraso (msgs)	Problemas
Centralizado	3	2	falha do coordenador
Descentralizado	$3 \times m \times k$, sendo k o número de tentativas	$2 \times m$	Starvation, baixa eficiência
Distribuído	$2 \times (n - 1)$	$2 \times (n - 1)$	Falha de qualquer processo
Token ring	1 até ∞	0 até $n - 1$	Falha de processo, perda de token

Resumo

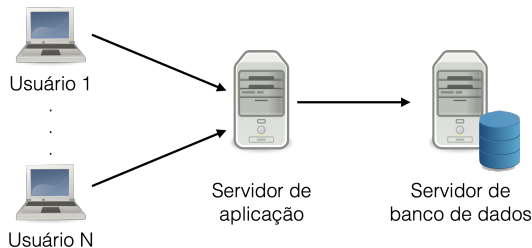
- Com exceção do algoritmo descentralizado, todos os demais estão susceptíveis a falhas
 - Algoritmo distribuído é até pior que o centralizado
- Todos os algoritmos não são adequados para sistemas tolerante a falhas
 - Se falhas não forem frequentes, então pode-se adotá-los
- Apesar do descentralizado ser tolerante a falhas, este precisa de melhorias para evitar a postergação indefinida e para melhorar seu desempenho

Algoritmo de eleição

Banco de dados distribuídos

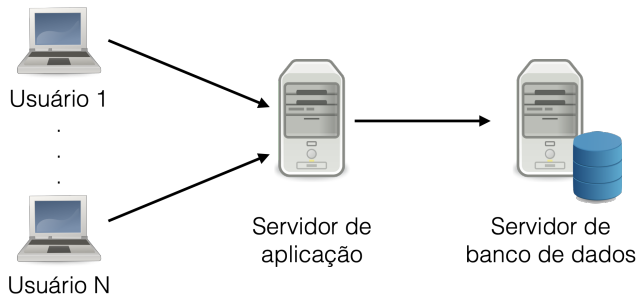
Exemplo

- Guarda informações de maneira estruturada e apresenta forma eficiente para acesso e modificação
- Sistemas de informação são clientes de sistemas de banco de dados



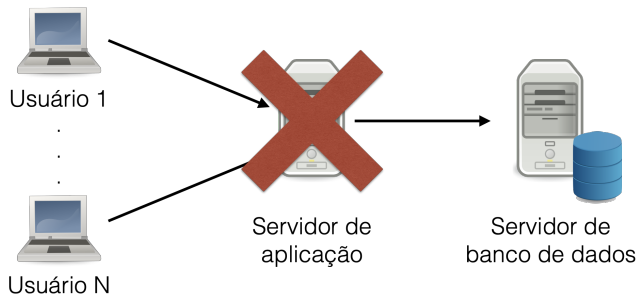
Banco de dados distribuídos

Como ter uma aplicação adequada para ambientes de larga escala?



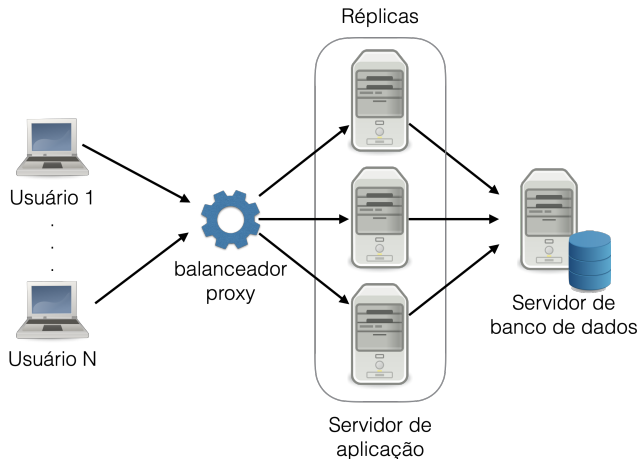
Banco de dados distribuídos

Como ter uma aplicação adequada para ambientes de larga escala?



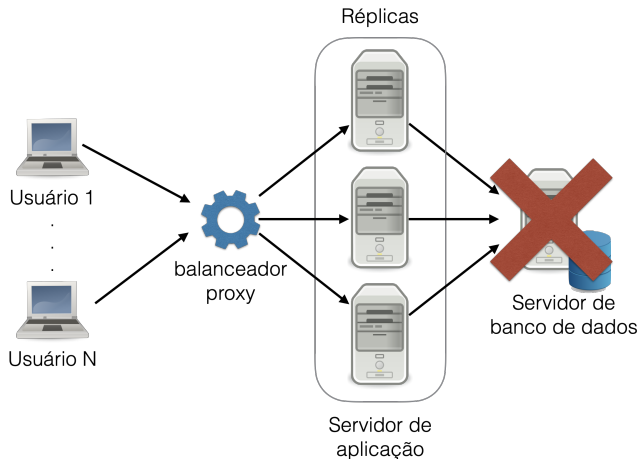
Banco de dados distribuídos

Como ter uma aplicação adequada para ambientes de larga escala?



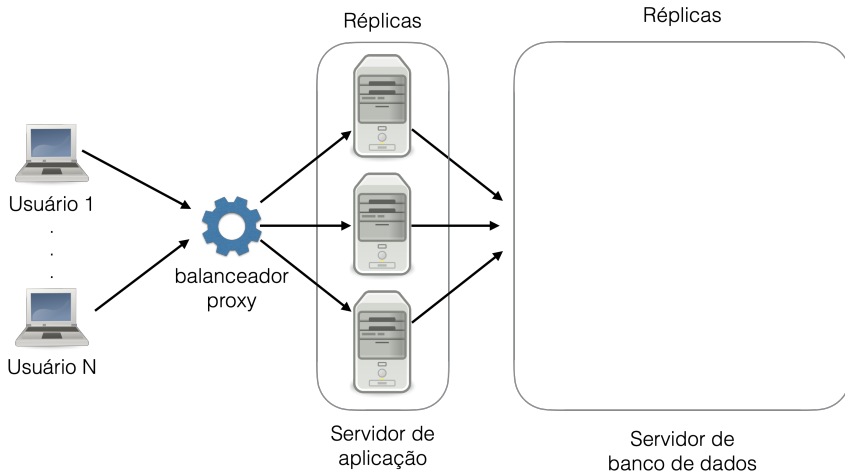
Banco de dados distribuídos

Como aumentar a segurança e a disponibilidade dos dados armazenados?



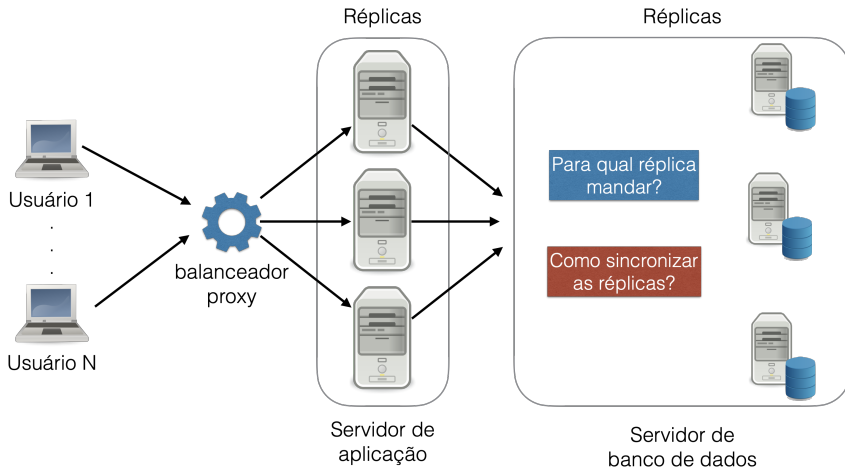
Banco de dados distribuídos

Como aumentar a segurança e a disponibilidade dos dados armazenados?



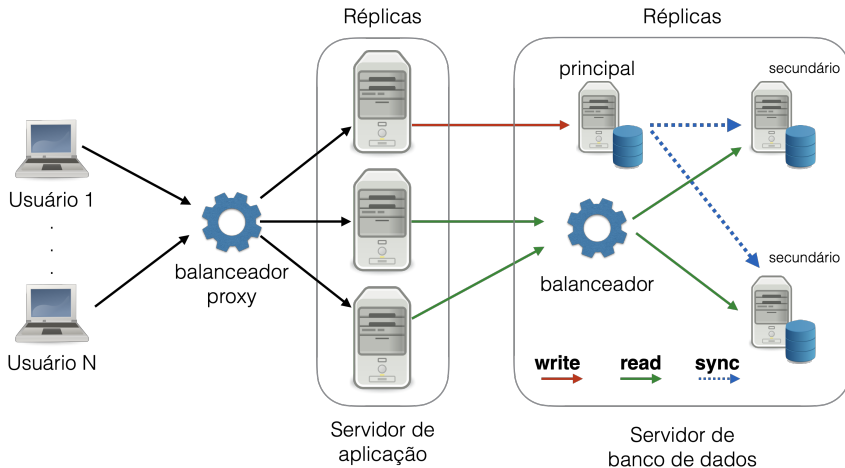
Banco de dados distribuídos

Como aumentar a segurança e a disponibilidade dos dados armazenados?



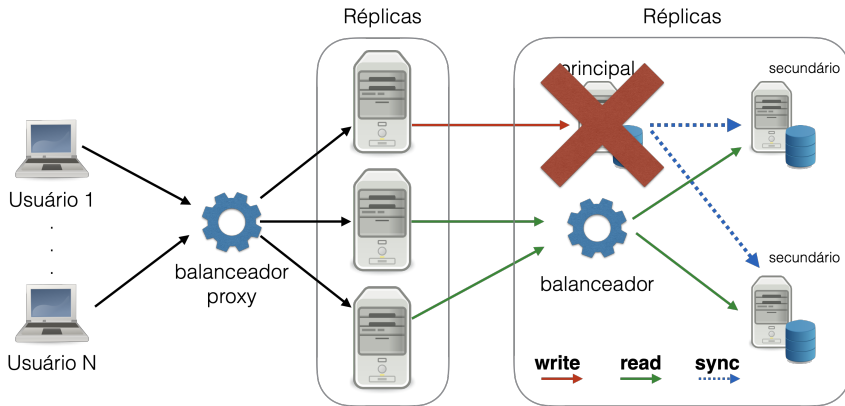
Banco de dados distribuídos

Como aumentar a segurança e a disponibilidade dos dados armazenados?



Banco de dados distribuídos

Como aumentar a segurança e a disponibilidade dos dados armazenados?



Como determinar quem será o novo principal?

Algoritmo de eleição

Necessidade de um coordenador em alguns algoritmos

- Em muitos sistemas, o coordenador é definido de forma estática e manual (por exemplo, um servidor de arquivos). Essa abordagem é centralizada e cria um ponto único de falha
- Como selecionar o coordenador de maneira dinâmica e automática?

Premissas para algoritmos de eleição

- Cada processo possui um identificador único.
- Todos os processos conhecem os identificadores dos demais.
- Nenhum processo sabe previamente quais processos estão ativos ou inativos.
- Após o início da eleição, todos os processos concordarão sobre quem será o novo coordenador.

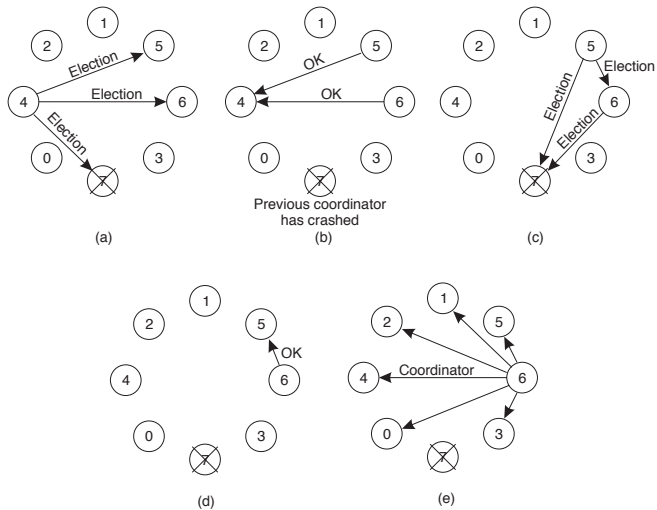
Algoritmo do Ditador – 1982

Quando um processo P_i detecta que o coordenador não está mais respondendo (*timeout*), ele inicia uma eleição.

- 1 P_i envia uma mensagem ELEIÇÃO para todos os processos com identificadores maiores que o seu.
- 2 Se não houver resposta, P_i se declara o novo coordenador.
- 3 Se algum processo com identificador maior responder, P_i desiste da eleição.
 - O processo que respondeu envia uma mensagem OK.
- 4 O processo que se torna o novo líder (ou seja, não recebeu mensagens OK) envia uma mensagem COORDENADOR para todos os demais processos.
- 5 Ao receber a mensagem COORDENADOR, cada processo define o emissor como o novo líder eleito.

Algoritmo do Ditador – 1982

Processo 4 detecta que o coordenador (P_7) não está mais respondendo



Outros algoritmos e softwares relacionados

- **Paxos – protocolo de consenso distribuído**

- [https://en.wikipedia.org/wiki/Paxos_\(computer_science\)](https://en.wikipedia.org/wiki/Paxos_(computer_science))

- **Raft – algoritmo para consenso distribuído**

- <https://raft.github.io/>

- **Apache ZooKeeper – ZooKeeper Atomic Broadcast (ZAB) protocol**

- <https://zookeeper.apache.org/>

- **Como a Amazon AWS elege um líder**

- <https://aws.amazon.com/pt/builders-library/leader-election-in-distributed-systems/>

- *“Evitamos depender do tempo em sistemas distribuídos... É difícil garantir que os relógios do sistema em um cluster estejam sincronizados o suficiente para depender dessa sincronização para ordenar ou coordenar operações distribuídas”*

Transações em sistemas distribuídos

O que são Transações?

- **Transação:** sequência de operações que garante a integridade dos recursos, mesmo com acessos concorrentes
- **Execução:**
 - 1 Inicia a transação
 - 2 Executa operações sobre os recursos
 - 3 Finaliza a transação com:
 - *commit*: salva as alterações
 - *abort*: desfaz as alterações

Todas as operações da transação são executadas integralmente ou nenhuma é aplicada

Propriedades das Transações (ACID)

- **Atômica:** executa tudo ou nada
- **Consistente:** leva o sistema de um estado válido a outro
- **Isolada:** transações concorrentes não interferem entre si
- **Durável:** após *commit*, as alterações são permanentes

Exemplo: transferência bancária

```
public transferirDinheiro(Conta origem, Conta destino, double quantia){  
    beginTransaction();  
    if ((origem.getSaldo() - quantia) >= 0){  
        origem.debita(quantia);  
        destino.credita(quantia);  
        commitTransaction();  
    } else {  
        abortTransaction();  
    }  
}
```

- Garante que a transferência seja atômica: ou ambas as contas são atualizadas, ou nenhuma alteração ocorre
- Se não houver saldo suficiente, a transação é abortada
- Falhas durante a transação também resultam em reversão das alterações

Desafio das transações distribuídas

Problema

Quando um banco de dados está distribuído em várias máquinas, é necessário garantir que todas as réplicas concordem sobre o resultado de uma transação: ou todas persistem as alterações (*commit*), ou todas as descartam (*abort*)

- Como garantir que nenhuma réplica fique em um estado diferente das demais, mesmo diante de falhas de rede ou de máquinas?
- Como evitar que parte do sistema considere a transação concluída enquanto outra parte a descarta?

Exemplo: transferência bancária distribuída

- Imagine uma transferência entre contas em bancos diferentes, cada um rodando em servidores distintos
- O débito ocorre em um servidor, o crédito em outro
- O que acontece se, após debitar, o servidor do crédito falha antes de confirmar a operação?

Exemplo: transferência bancária distribuída

- Imagine uma transferência entre contas em bancos diferentes, cada um rodando em servidores distintos
- O débito ocorre em um servidor, o crédito em outro
- O que acontece se, após debitar, o servidor do crédito falha antes de confirmar a operação?

Consequências

- Se apenas o débito for persistido, o dinheiro “desaparece”
- Se apenas o crédito for persistido, dinheiro é “criado”
- Precisamos garantir que ambos os lados concordem sobre o sucesso ou fracasso da transação

Requisitos para transações distribuídas

■ Segurança (*safety*):

- Se uma réplica faz *commit*, nenhuma pode fazer *abort*
- Se uma réplica faz *abort*, nenhuma pode fazer *commit*

■ Vivacidade (*liveness*):

- Se não houver falhas, a decisão é tomada e a transação é concluída
- Se houver falhas, o sistema deve chegar a um acordo assim que possível

Solução: Protocolo de *commit* em duas fases (2PC)

Como garantir decisão atômica?

O protocolo 2PC (Two-Phase Commit) coordena as réplicas para que todas concordem sobre o resultado da transação, mesmo em ambientes distribuídos

- Um coordenador inicia o processo e consulta todas as réplicas
- Só após todas confirmarem que podem persistir, o coordenador ordena o *commit*
- Se alguma réplica não puder persistir, todas abortam

Protocolo de *commit* em duas fases (2PC) I

- O 2PC é um protocolo clássico para garantir que todas as réplicas de um sistema distribuído concordem sobre o resultado de uma transação (*commit* ou *abort*), mesmo diante de falhas.
- Existem dois papéis principais:
 - **Coordenador:** gerencia a transação
 - **Participantes:** réplicas que executam as operações

Protocolo de *commit* em duas fases (2PC) II

Fase 1 – Votação (*prepare*)

- 1 O coordenador envia uma mensagem PREPARE para todos os participantes, solicitando que preparem a transação
- 2 Cada participante tenta executar a transação localmente, registra a intenção em um log não volátil (*write-ahead log*) e responde com VOTE_COMMIT (se puder prosseguir) ou VOTE_ABORT (em caso de erro)

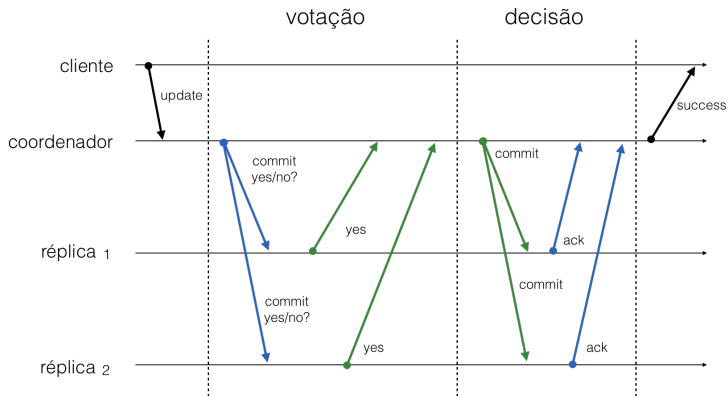
Protocolo de *commit* em duas fases (2PC) III

Fase 2 – Decisão (*commit/abort*)

- 1 Se todos os participantes responderem VOTE_COMMIT, o coordenador envia GLOBAL_COMMIT; caso contrário, envia GLOBAL_ABORT
- 2 Cada participante, ao receber a decisão, aplica ou descarta as alterações e registra o resultado em log permanente

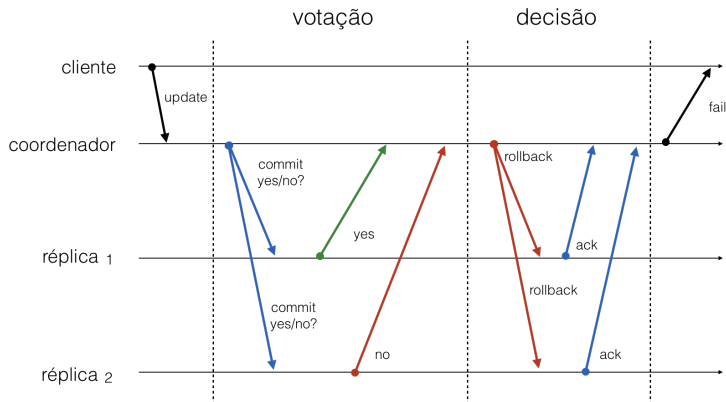
Observação: O 2PC garante atomicidade, mas pode bloquear participantes em caso de falha do coordenador.

Fluxo do protocolo 2PC



- **Commit:** todos os participantes concordam, transação é efetivada
- **Abort:** qualquer participante rejeita, todos abortam a transação

Fluxo do protocolo 2PC

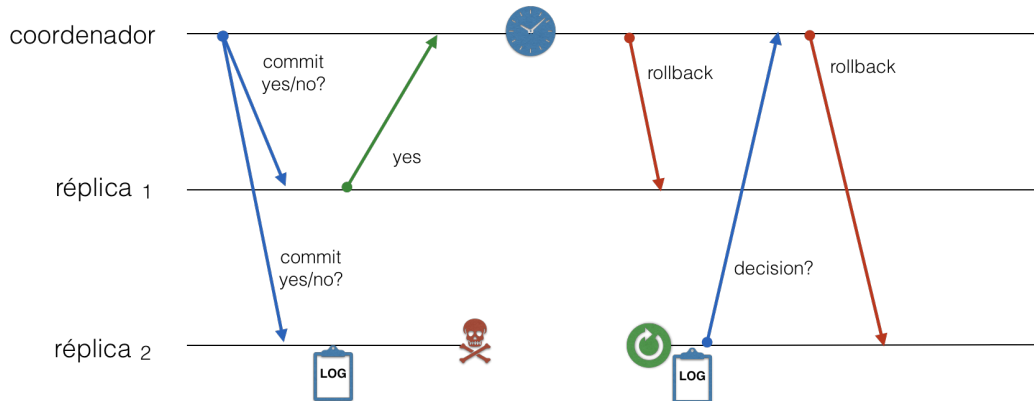


- **Commit:** todos os participantes concordam, transação é efetivada
- **Abort:** qualquer participante rejeita, todos abortam a transação

Limitações e desafios do 2PC

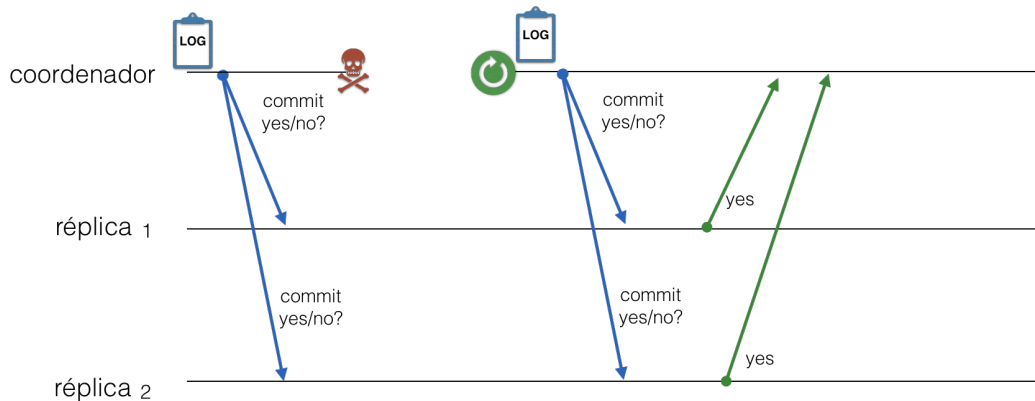
- **Bloqueio do coordenador:** Se o coordenador falhar após receber votos, participantes ficam bloqueados aguardando a decisão final
- **Bloqueio dos participantes:** Se um participante falhar após votar, pode atrasar a decisão global
- **Soluções parciais:**
 - Coordenador pode definir *timeout* e decidir por GLOBAL_ABORT se não receber todos os votos
 - Participantes podem consultar outros participantes para tentar descobrir a decisão, mas se todos aguardam, o sistema permanece bloqueado até a recuperação do coordenador
- **Conclusão:** O 2PC é seguro, mas não tolera falhas de forma eficiente (pode causar bloqueio indefinido)

Cenários de falha no 2PC



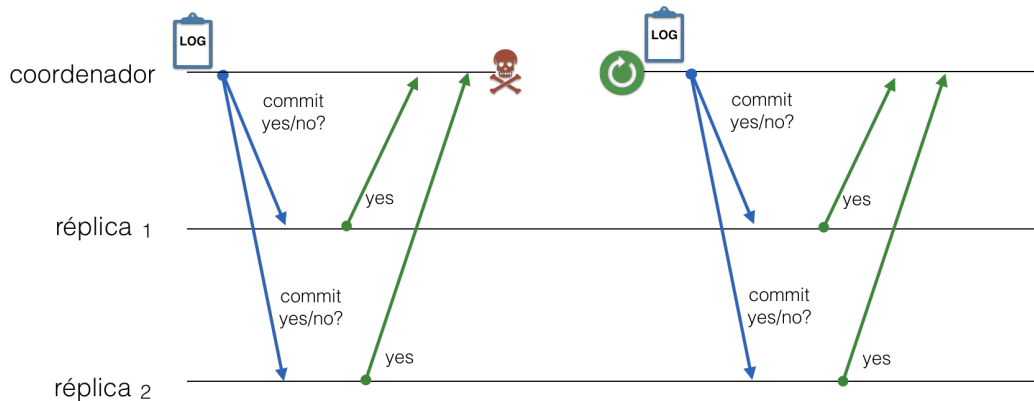
- **Falha de participante:** Réplica falha antes de votar; coordenador pode abortar após timeout

Cenários de falha no 2PC



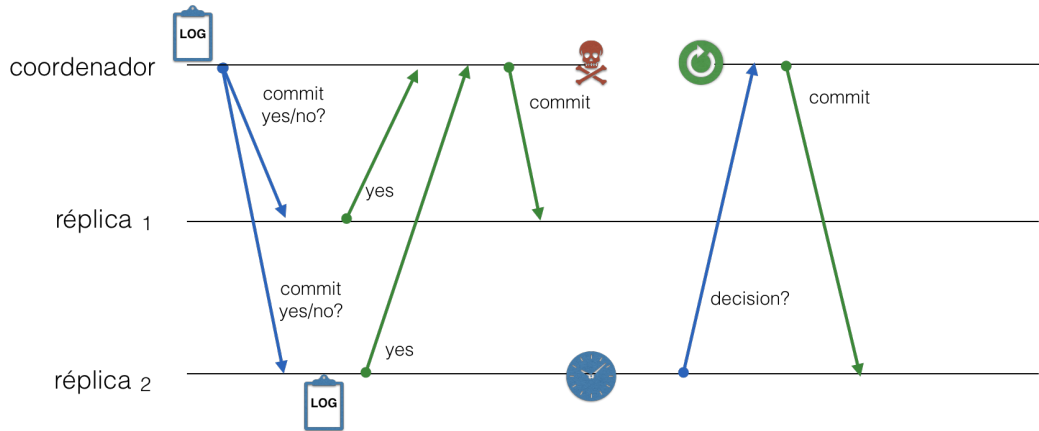
- **Falha do coordenador (antes dos votos):** Participantes não sabem a decisão, aguardam recuperação

Cenários de falha no 2PC



- **Falha do coordenador (após votos, antes da decisão):** Participantes ficam bloqueados, aguardando decisão

Cenários de falha no 2PC



- **Falha do coordenador (durante envio da decisão):** Alguns participantes podem aplicar a decisão, outros ficam bloqueados

Consenso em *Blockchain*

- *Blockchains* exigem que todos os nós concordem sobre a inclusão de novos blocos, garantindo uma única versão da cadeia
- O consenso é alcançado por algoritmos como *Proof of Work*, *Proof of Stake* ou protocolos tolerantes a falhas bizantinas
- Esses mecanismos permitem que o sistema funcione mesmo com falhas ou participantes maliciosos, sem coordenador central
- O objetivo é garantir consistência global e evitar visões divergentes do estado do sistema

Diferente do 2PC, que depende de um coordenador e pode bloquear em caso de falhas, algoritmos de consenso em *blockchain* são projetados para tolerar falhas e não possuem ponto único de bloqueio

Aulas baseadas em

Algumas figuras e textos



TANENBAUM, Andrew S; STEEN, Maarten van. **Sistemas Distribuidos: Princípios e paradigmas**. 2. ed.: Prentice Hall, 2008.